

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 8

STRUCTURED QUERY LANGUAGE (SQL):

DATA MANIPULATION LANGUAGE (DML)

STRUCTURED QUERY LANGUAGE (SQL): **OUTLINE**

SQL DML

SQL DDL

BANK RELATIONAL SCHEMA

Account(accountNo, balance, *branchName*)

Borrower(clientId, loanNo)

Branch(branchName, district, liabilities, assets)

Client(clientId, name, hkid, address, district, rating)

Depositor(clientId, accountNo)

Loan(loanNo, amount, year, *branchName*)

Tags(clientId, tag)

Attribute names in italics are foreign key attributes.

BANK RELATIONAL DATABASE

Client

<u>clientId</u>	name	hkid	address	district	rating
3	Steven Chan	C835618(4)	Mainmount Road	Central and Western	9
4	Mary Kwan	M534109(3)	Hing Wah Street	Islands	6
8	Isaac Li	K917263(6)	Man Lai Court	null	6
17	Pam Du	W251984(6)	Tsing Yi Road	null	8

Tags

<u>clientId</u>	tag
3	vip
3	senior
4	senior
17	vip

Depositor

<u>clientId</u>	<u>accountNo</u>
3	A-117
3	A-120
3	A-301
4	A-215
4	A-322
8	A-120

Branch

<u>branchName</u>	district	liabilities	assets
Diamond Hill	Kwun Tong	55000	0
Festival Walk	Yau Tsim Mong	0	107000
Pacific Place	Central and Western	120000	40000
Star House	Yau Tsim Mong	115000	115000

Borrower

<u>clientId</u>	<u>loanNo</u>
3	L-171
3	L-435
4	L-321
4	L-434
17	L-435

Account

<u>accountNo</u>	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

Loan

<u>loanNo</u>	amount	year	branchName
L-171	115000	2024	Star House
L-321	40000	2024	Pacific Place
L-434	52000	2025	Festival Walk
L-435	55000	2025	Festival Walk

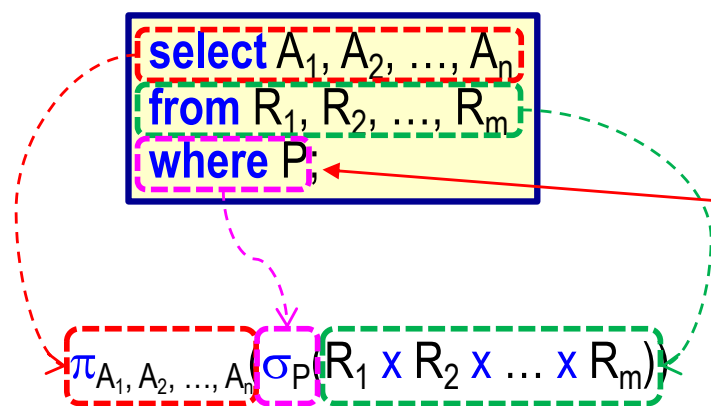
Attribute names in italics
are foreign key attributes.

STRUCTURED QUERY LANGUAGE (SQL)

- **SQL** is used in all commercial relational database systems.
 - It is an American National Standards Institute (ANSI) standard.
 - Not all relational database systems implement all language features of the ANSI standard or implement them according to the ANSI standard.
 - Many relational database systems also include additional language features that are not part of the ANSI standard.
- **SQL** is composed of a
 - **data definition language** (DDL) used to specify relation schemas.
 - **data manipulation language** (DML) used to query and change data in a relational database.

SQL DML BASIC STRUCTURE

- **SQL DML** is based on **set** and **relational algebra** operations with certain **modifications** and **enhancements**.
- An **SQL** query has the basic form:



A_i are **attributes**

R_i are **relations**

P is a **predicate** (condition)

An **SQL** query ends with a semicolon.

The equivalent relational algebra expression.

- An **SQL** query result is a relation (**but it may contain duplicates**).
- ✎ **SQL queries can be composed (nested).**

PROJECTION: SELECT CLAUSE

- The **select** clause corresponds to the relational algebra **projection** (π) operation; it specifies attributes to include in the query result.

Query: Find the districts of clients.

select district
from Client;

Query result

district
Central and Western
Islands
null
null

≠

$\pi_{\text{district}}(\text{Client})$



SQL does not remove duplicates in the query result by default.

- An asterisk (*) in the **select** clause denotes “**all attributes**”.

Query: Find all client tuples.

select *
from Client;

=

select clientId, name, hkid, address, district, rating
from Client;

Attributes specified in the **select clause must be defined in the relations in the **from** clause.**

Client(clientId, name, hkid, address, district, rating)



PROJECTION: DUPLICATE REMOVAL

- The keyword **distinct** forces the **removal of duplicates**.

Query: Find the unique districts of all clients.

```
select distinct district  
from Client;
```

Query result

district
Central and Western
Islands
null

=

```
 $\pi_{\text{district}}(\text{Client})$ 
```

- The keyword **all** specifies that **duplicates should not be removed** (needed for some set operations—see later lecture notes).

Query: Find the districts of all clients retaining duplicates.

```
select all district  
from Client;
```

force the DBMS not to remove duplicates
(same as omitting **all**)

PROJECTION: ARITHMETIC OPERATIONS

- The **select** clause can contain arithmetic expressions involving the math operators $+$, $-$, $\div$ and $\times$ that can operate on constants or attributes of tuples.

Query: Multiply the amount of each loan by 100.

```
select loanNo, amount*100 branchName  
from Loan;
```

The query result is a relation that contains the same data as the **Loan** relation, except that the **amount** values are multiplied by 100.

Query result

loanNo	amount*100	branchName
L-171	11500000	Star House
L-321	4000000	Pacific Place
L-434	5200000	Festival Walk
L-435	5500000	Festival Walk

 The database (i.e., the **Loan** relation) is not changed, only the query result is changed.

SELECTION: WHERE CLAUSE

- The **where** clause corresponds to the relational algebra **selection predicate** (σ) and specifies the criteria that tuples in the relations in the **from** clause must satisfy.

Query: Find all clients who live in Islands district.

```
select *
from Client
where district='Islands';
```

Query result

clientId	name	hkid	address	district	rating
4	Mary Kwan	M534109(3)	Hing Wah Street	Islands	6

What is the equivalent
relational algebra expression?

👉 Attributes specified in a **where** clause **must** be defined in the relations in the **from** clause.

SQL Note

String values **must** be enclosed in single quotes.
Numeric values **do not** require quotes (and it is recommended **not** to use quotes).

Client(clientId, name, hkid, address, district, rating)

SELECTION: COMPARISON OPERATORS

- Comparison operators that can be used in a **where** clause are:

=	equal	>	greater than	<	less than
>=	greater or equal	<=	less or equal	<> or !=	not equal

Query: Find the names and districts of all clients who do not live in Islands district.

```
select name, district
from Client
where district<>'Islands';
```

```
select name, district
from Client
where district!='Islands';
```

Query result

name	district
Steven Chan	Central and Western

SQL Note

Tuples whose value is **null** for the specified attribute are normally **not** returned in the query result.

Query: Find the loans whose amount is more than \$50,000.

```
select *
from Loan
where amount>50000;
```

Query result

loanNo	amount	year	branchName
L-171	115000	2024	Star House
L-321	52000	2024	Pacific Place
L-435	55000	2025	Festival Walk

Client(clientId, name, hkid, address, district, rating)

Loan(loanNo, amount, year, branchName)



SELECTION: BETWEEN OPERATOR

- For convenience, the **between** comparison operator can be used when a range of values should be matched.

Query: Find the name and rating of clients whose rating is between 7 and 9 (i.e., ≥ 7 and ≤ 9).

```
select name, rating
from Client
where rating between 7 and 9;
```

Query result

name	rating
Steven Chan	9
Pam Du	8

SQL Note

The range includes the endpoints.

- Can also use **not between**.

Query: Find the id and rating of clients whose rating is not between 7 and 9 (i.e., < 7 or > 9).

```
select name, rating
from Client
where rating not between 7 and 9;
```

Query result

name	rating
Mary Kwan	6
Isaac Li	6

SQL Note

The negation includes the endpoints.

Client(clientId, name, hkid, address, district, rating)

SELECTION: BOOLEAN OPERATORS

- Boolean operators **and**, **or** and **not** as well as arithmetic expressions can be used in a **where** clause.

Query: Find the name of clients who live in Islands district whose credit rating is less than 8.

```
select name
from Client
where district='Islands' and rating<8;
```

Query result

name
Mary Kwan

- The **and** operator has higher precedence than the **or** operator.

Query: Find the name of clients who live in Central and Western district or Islands district and whose credit rating is less than 8.

```
select name
from Client
where (district='Central and Western' or district='Islands')
and rating<8;
```

Query result

name
Mary Kwan

Client(clientId, name, hkid, address, district, rating)



SELECTION: NULL COMPARISON

- How to find tuples where the value of an attribute is **null**?

Query: Find the name of clients whose district value is null.

~~select name
from Client
where district=null;~~

Will not match null values!
(Returns no query result.)



select name
from Client
where district **is null**;

Query result

name
Isaac Li
Pam Du

SQL Note

In **SQL**, **null** can mean that the attribute value is

1. missing (i.e., unknown) *or*
2. not applicable to this instance.

☞ Since **null** is a special value, comparisons to **null** are specified differently in **SQL** than comparisons to other values.

How would you specify
to find clients whose
district value is not null?

Client(clientId, name, hkid, address, district, rating)

SELECTION: NULL REPLACEMENT

The **coalesce** function can be used to replace **null** values in a query result.

- The function takes one or more arguments and returns the first non-null argument or **null** if all arguments are **null**.
- The arguments must all be of the **same data type**.

Query: Find the name and district of all clients; replace null district values with 'unknown'.

```
select name, coalesce(district, 'unknown')  
from Client;
```

👉 **Normally, only two arguments are specified.**

Query result

name	district
Steven Chan	Central and Western
Mary Kwan	Islands
Isaac Li	unknown
Pam Du	unknown

Oracle Note

Oracle also provides the function **nvl** to replace null values in a query result. Unlike the **SQL coalesce** function, the **Oracle nvl** function takes only two arguments and, moreover, the two arguments can be of the same or different data types. If their data types are different, one is implicitly converted to the other, if possible,

CARTESIAN PRODUCT: FROM CLAUSE

- The **from** clause corresponds to the relational algebra **Cartesian-product operation** ($\times$).

Query: Find the Cartesian product of borrower and loan.

```
select *
from Borrower, Loan;
```

This can also be specified as

```
select *
from Borrower cross join Loan;
```

👉 **A from clause with more than one relation is rarely used without a join condition.**

Loan(loanNo, amount, year, branchName)

Borrower(clientId, loanNo)

Query result

clientId	loanNo	loanNo_1	amount	year	branchName
3	L-171	L-171	115000	2024	Star House
3	L-171	L-321	52000	2024	Pacific Place
3	L-171	L-434	40000	2025	Festival Walk
3	L-171	L-435	55000	2025	Festival Walk
3	L-435	L-171	115000	2024	Star House
3	L-435	L-321	52000	2024	Pacific Place
3	L-435	L-434	40000	2025	Festival Walk
3	L-435	L-435	55000	2025	Festival Walk
4	L-32	L-171	115000	2024	Star House
4	L-321	L-321	52000	2024	Pacific Place
⋮					
17	L-435	L-321	40000	2024	Pacific Place
17	L-435	L-434	52000	2025	Festival Walk
17	L-435	L-435	55000	2025	Festival Walk



NATURAL JOIN: FROM CLAUSE

- A **natural join** is specified using the keywords **natural join** in the **from** clause.

Query: Find the client id and loan numbers of the clients with loans.

```
select name, loanNo
from Borrower natural join Loan;
```

Query result

clientId	loanNo
3	L-171
3	L-435
4	L-321
4	L-434
17	L-435

- Multiple relations can be joined in a **from** clause.

Query: Find the name and loan numbers of the clients with loans.

```
select name, loanNo
from Client natural join Borrower natural join Loan;
```

Query result

name	loanNo
Steven Chan	L-171
Steven Chan	L-435
Mary Kwan	L-321
Mary Kwan	L-434
Pam Du	L-435

Client(clientId, name, hkid, address, district, rating)

Loan(loanNo, amount, year, branchName)

Borrower(clientId, loanNo)

NATURAL JOIN: FROM CLAUSE (CONTD)

- A natural join will join on all the common attributes of the two relations.
- Consider the following relation schema and query:

Author(authorId, title, name)

Book(bookId, title, authorId)

Query: Find the names of authors.

~~select name
from Author natural join Book;~~

This query joins the relations on both authorId and title!

- A **using** clause can specify on which attributes to perform a natural join when not all the common attributes should be used.

select name
from Author join Book using (authorId);

SQL Note

The keyword **natural** is not allowed with the **using** clause.

θ -JOIN/EQUIJOIN: FROM/WHERE CLAUSE

- A θ -join or equijoin can be specified by adding the appropriate join condition in the **from** clause with an **on** clause.

Query: Find the client id, loan number, loan amount, year and branch name of the clients who have loans.

```
select clientId, Loan.loanNo, amount, year, branchName
from Loan join Borrower on Loan.loanNo=Borrower.loanNo;
```

SQL Note

Attribute names must be qualified if ambiguous.

**Query
result**

clientId	loanNo	amount	year	branchName
3	L-171	115000	2024	Star House
3	L-435	55000	2025	Festival Walk
4	L-321	40000	2024	Pacific Place
4	L-434	52000	2025	Festival Walk
17	L-435	55000	2025	Festival Walk

Join attribute names cannot be qualified in a natural join.

Why?

The join condition can also be specified in a **where** clause.

```
select clientId, Loan.loanNo, amount, year, branchName
from Loan, Borrower
where Loan.loanNo=Borrower.loanNo;
```

Loan(loanNo, amount, year, *branchName*)

Borrower(*clientId*, loanNo)

OUTER JOIN: FROM CLAUSE

- An **outer join** is specified using the keywords [**natural**] {**full** | **left** | **right**} **outer join** in the **from** clause.

Query: Find the client id, loan number and district of clients; include also clients who have no loan.

```
select clientId, loanNo, district
from Client natural left outer join Borrower;
```

The join condition can also be specified in an **on** clause.

```
select Client.clientId, loanNo, district
from Client left outer join Borrower
on Client.clientId=Borrower.clientId;
```

Query result

clientId	loanNo	district
3	L-171	Central and Western
3	L-435	Central and Western
4	L-321	Islands
4	L-434	Islands
8	null	null
17	L-435	null

SQL Note

An outer join join-condition cannot be specified in a **where** clause.

What is the difference in the result of these two queries?

SET OPERATIONS: UNION, INTERSECT, MINUS

- The operations **union**, **intersect**, and **minus** operate on relations and correspond to the relational algebra operations $\cup$, $\cap$ and $-$.

SQL Note

The keyword **minus** is **Oracle** specific.
The SQL standard uses the keyword **except** rather than **minus**.

- Each of the set operations **automatically removes duplicates**.
- Specifying the keyword **all** keeps all duplicates.

Oracle Note

Only **union all** is supported.

SET OPERATIONS: EXAMPLES

Query: Find the id of clients who have a loan, an account, or both.

```
select clientId from Depositor
union
select clientId from Borrower;
```

Query result

clientId
3
4
8
17

Query: Find the id of clients who have both an account and a loan.

```
select clientId from Depositor
intersect
select clientId from Borrower;
```

Query result

clientId
3
4

Query: Find the id of clients who have an account, but do not have a loan.

```
select clientId from Depositor
minus
select clientId from Borrower;
```

Query result

clientId
8

Depositor(clientId, accountNo)



Borrower(clientId, loanNo)

RENAME ATTRIBUTES: AS CLAUSE

- Attributes can be renamed using the **as** clause:

old-name as new-name

Query: Find the name and loan number of all clients having a loan at the Pacific Place branch; replace the attribute name loanNo with the name loanId.

```
select distinct clientId, loanNo as loanId
from Borrower natural join Loan
where branchName= 'Pacific Place';
```

Query result

clientId	loanId
4	L-321

```
select distinct clientId, loanNo loanId
from Borrower natural join Loan
where branchName= 'Pacific Place';
```

Oracle Note

The keyword **as** *is optional* in the **select** clause.

Loan(loanNo, amount, year, *branchName*)

COMP 3311



Borrower(*clientId*, loanNo)

LECTURE 8
SQL DML

23

RENAME RELATIONS: TABLE ALIAS

- Renaming a relation is convenient for replacing a long relation name used multiple times in a query with a shorter one (i.e., with an alias).

Query: Find the client name and loan number for clients that have a loan at *some* branch; replace the attribute name loanNo with the name loanId.

```
select distinct clientId, B.loanNo loanId
from Borrower B, Loan L
where B.loanNo=L.loanNo;
```

A relation identifier (such as B and L) is referred to as a *correlation name* in SQL.

➤ Also known as *table alias*, *correlation variable* or *tuple variable*.

Query result

clientId	loanId
3	L-171
3	L-435
4	L-321
4	L-434
17	L-435

Oracle Note

Relations in the **from** clause are renamed using an identifier without the keyword **as**.

- While the SQL standard renames relations in the **from** clause using the **as** keyword, the **as** keyword is not allowed in Oracle.



RENAME RELATIONS: TABLE ALIAS (CONTD)

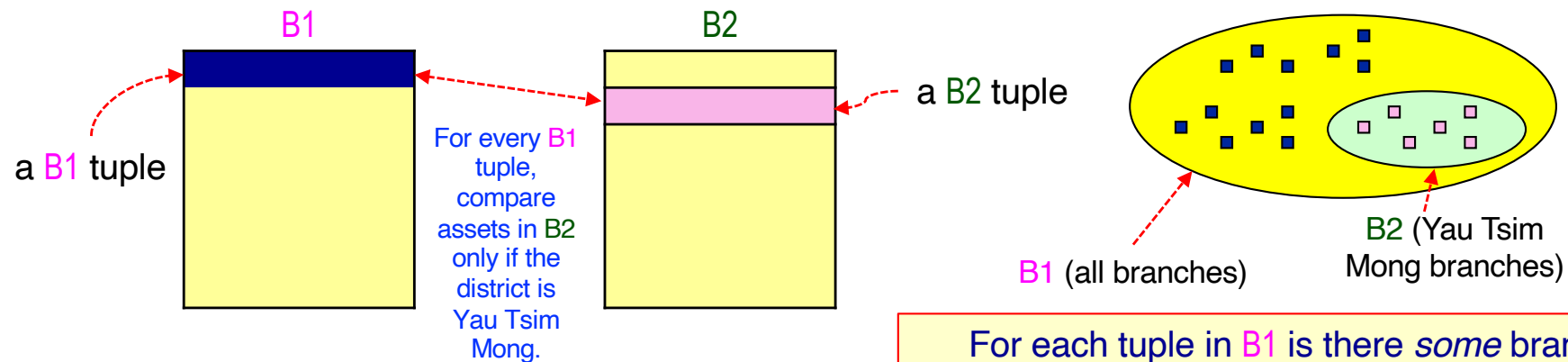
- Renaming a relation is required when we want to **compare tuples in the same relation** (e.g., when doing a self-join).

Query: Find the name of branches that have greater assets than some (i.e., at least one) branch located in Yau Tsim Mong.

```
select distinct B1.branchName
from Branch B1, Branch B2
where B1.assets > B2.assets and B2.district = 'Yau Tsim Mong';
```

Query result

branchName
Star House



For each tuple in B1 is there some branch located in Yau Tsim Mong that has less assets? If yes, add the B1 tuple to the query result.

Branch(branchName, district, assets)



STRING PATTERN MATCHING: **LIKE** OPERATOR

- The **like** operator is used to match characters in string values.
- String values also can be compared to a pattern using the special pattern matching characters:
 - %** (percent) matches any substring.
 - _** (underscore) matches any single character.

Query: Find the name and address of all clients whose address includes the substring 'Main' (e.g., Main Street, Mainmount Road, Mainly Avenue, ...).

```
select name, address
from Client
where address like '%Main%'
```

Query result

name	address
Steven Chan	Mainmount Road

 **Pattern matching is *normally* case-sensitive.**

STRING PATTERN MATCHING: **LIKE OPERATOR**

(CONTD)

- To match the special pattern matching characters in a string, **SQL** allows the specification of an escape character.
 - Suppose we use backslash (\) as the escape character.
 - **like** '20\%%' **escape** '\' matches all strings beginning with “20%”
 - **like** 'pair_%' **escape** '\' matches all strings beginning with “pair_”
- To match a single quote in a string, **use two single quotes**.
 - **like** 'Toms"s%' matches all strings beginning with “Tom's”

STRING PATTERN MATCHING: REGEXP_LIKE OPERATOR

- The **regexp_like** operator is used to match regular expressions.

Query: Find the name of those clients whose names begin with Steven or Stephen (i.e., the name begins with 'Ste' followed by either 'v' or 'ph' followed by 'en' followed by any other characters).

```
select name
from Client
where regexp_like (name, '^Ste(v|ph)en');
```

Query result

name
Steven Chan

Regexp Note

The character **^** matches the start position of a string.

The character **\$** matches the end position of a string.

Query: Find the name of those clients with a double vowel (i.e., double a, e, i, o or u) in their name, regardless of case.

```
select name
from Client
where regexp_like (name, '([aeiou])\1', 'i');
```

Query result

name
Isaac Li

Regexp Note

The **\1** in the pattern string is called a backreference. It references what is captured by the string in () and tries to match it again.

Client(clientId, name, hkid, address, district, rating)



STRING PATTERN MATCHING: REGEXP_LIKE OPERATOR (CONTD)

Usage: `regexp_like`(*source_string*, *pattern*, [*match_parameter*])

where:

- *source_string* is a search value (usually an attribute name);
- *pattern* is a regular expression;
- *match_parameter* specifies a matching behaviour as follows
 - 'i' specifies case-insensitive matching.
 - 'c' specifies case-sensitive matching.
 - 'n' allows the period (.), which is normally the match-any-character wildcard character, to match the newline character.
 - 'm' treats the source string as multiple lines.

If *match_parameter* is omitted, then:

- The default case sensitivity is used (usually case-sensitive).
- A period (.) does not match the newline character.
- The source string is treated as a single line.

For information on the regular expressions supported by Oracle see https://docs.oracle.com/cd/B12037_01/server.101/b10759/ap_posix001.htm#690819.

For examples of regular expression use in Oracle see <https://www.salvis.com/blog/2018/09/28/regular-expressions-sql-examples/>.

For testing your regular expressions see <https://www.regextester.com/> (use the PCRE option).

ORDERING RESULT TUPLES: ORDER BY CLAUSE

Query: Find the client id, name and rating for clients who have a loan in 2025. Order the result first by credit rating descending and then by name ascending.

```
select Client.clientId, name, rating
from Client, Borrower, Loan
where Client.clientId=Borrower.clientId
      and Borrower.loanNo=Loan.loanNo
      and year='2025'
order by rating desc, name;
```

Query result

clientId	name	rating
3	Steven Chan	9
17	Pam Du	8
4	Mary Kwan	6

Ordering options

asc - ascending (the default)

desc - descending

SQL Note

By default, a null value is the largest value in sort order. Thus, null values appear after all values in ascending sort order and before all values in descending sort order. The default ordering can be changed by specifying **nulls last** or **nulls first** for each attribute in an **order by** clause.

Client(clientId, name, hkid, address, district, rating)

Loan(loanNo, amount, year, branchName)

Borrower(clientId, loanNo)



LIMITING RESULT TUPLES: **FETCH CLAUSE**

- The **Oracle fetch** clause limits the number of rows retrieved by a query.

Query: Find the name and credit rating of any three clients with the highest credit ratings.

```
select name, rating
from Client
order by rating desc nulls last
fetch first 3 rows only;
```

Query result

name	rating
Steven Chan	9
Pam Du	8
Mary Kwan	6

Oracle Fetch Clause Note

Since null values are the largest value in sort order, it is necessary to specify **nulls last** to exclude any clients with a null rating.



Oracle Note

This syntax is Oracle specific.

LIMITING RESULT TUPLES: **FETCH CLAUSE** (CONT'D)

- The **with ties** clause returns additional rows that have the **same** sort key value as the last row fetched.

Query: Find the name and credit rating of any three clients with the highest credit ratings.

```
select name, rating
from Client
order by rating desc nulls last
fetch first 3 rows with ties
```

Query result

name	rating
Steven Chan	9
Pam Du	8
Mary Kwan	6
Isaac Li	6

Client(clientId, name, hkid, address, district, rating)



Oracle Note

This syntax is Oracle specific.

LIMITING RESULT TUPLES: FETCH CLAUSE (CONT'D)

- The **offset** clause specifies an optional number of rows (i.e., offset) to skip before the retrieval of rows starts.

Query: Find the third highest credit rating.

```
select distinct rating
from Client
order by rating desc nulls last
offset 2 rows
fetch next 1 row only;
```

Query result

rating
6

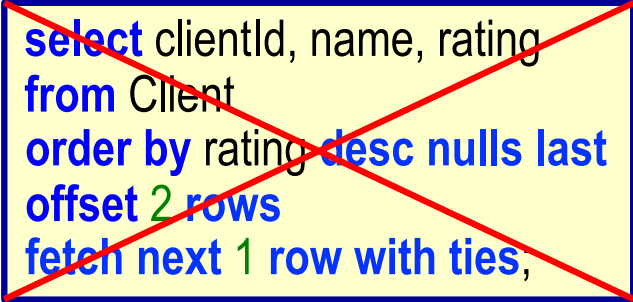


Oracle Note

This syntax is Oracle specific.

LIMITING RESULT TUPLES: FETCH CLAUSE (CONT'D)

Query: Find the client id, name and credit rating of all clients with the third highest credit rating. Order the result by client name ascending.



```
select clientId, name, rating  
from Client  
order by rating desc nulls last  
offset 2 rows  
fetch next 1 row with ties,
```

 **This query requires to first get the third highest credit rating and then get all the clients with this credit rating.**

(How to accomplish this in SQL is discussed in the next lecture.)

Client(clientId, name, hkid, address, district, rating)



LIMITING RESULT TUPLES: **FETCH CLAUSE** (CONT'D)

Usage: [**offset** *offset* **rows**]

fetch [**first** | **next**] [*row_count* | *percent percent*] [**row** | **rows**] [**only** | **with ties**]

where:

- **offset** specifies an optional number of rows (i.e., *offset*) to skip before the retrieval of rows starts.
- *row_count* specifies the number of rows to retrieve.
- *percent percent* specifies the percentage of rows to retrieve.
- **only** returns exactly the number of rows or percentage of rows specified.
- **with ties** returns additional rows that have the same sort key value as the **last row fetched** (requires an **order by** clause in the query).

☞ **While most relational database systems support limiting result tuples, the syntax may be different than that used in Oracle.**

CONDITIONAL LOGIC: CASE STATEMENT

The **case** statement evaluates a list of **when** clauses and returns the value of the first **when** clause that evaluates to true.

- If no **when** clause evaluates to true, the **else** clause value is returned.
- If neither the **else** clause nor any **when** clauses are true, **null** is returned.
- A **case** statement can be used **anywhere a value is expected** (e.g., it can be used in a predicate in a **where** clause).

Query: For each client categorize their credit rating as follows:
between 1 and 3, high risk; between 4 and 7, medium risk;
between 8 and 10, low risk otherwise unknown risk.

```
select name, case
    when rating <= 3 then 'high risk'
    when rating <= 7 then 'medium risk'
    when rating <= 10 then 'low risk'
    else 'unknown risk'
end as rating
from Client;
```

Query result

name	rating
Steven Chan	low risk
Mary Kwan	medium risk
Isaac Li	medium risk
Pam Du	low risk